

EPAM — Senior Software Engineer (Generative AI Engineer)

— Interview Prep

Duration: ~1.5 hrs technical | **Exp target:** 4+ yrs (3+ in GenAI/LLM/NLP)

How to use this doc: Interviewers at EPAM tend to go broad (breadth across the JD) then deep on 2-3 areas you claim strength in. For every answer below, tie it back to something you have actually built. When you don't know something, say "I haven't used X in production, but conceptually it does Y, and I'd approach it by Z." That scores higher than bluffing.

0. The 60-second self-introduction (they always start here)

"I'm a software engineer with X years of experience, and for the last ~3 years I've focused on building production GenAI systems. My core stack is Python and FastAPI for services, LangChain/LangGraph for orchestration, and RAG pipelines backed by vector DBs like FAISS and Pinecone. I've shipped [a RAG chatbot / document Q&A / agentic workflow] that [business outcome — e.g., cut support handling time 30%]. On the ops side I've handled deployment on [Azure/AWS], model versioning, evaluation, and CI/CD. What excites me about GenAI is turning research-y capabilities into robust, scalable, and cost-controlled products."

Keep it to 4 sentences. Have ONE flagship project you can talk about for 10 minutes.

PART 1 — Python & APIs

Q1. How do you design a production API for an LLM feature?

- **Framework:** FastAPI (async, Pydantic validation, auto OpenAPI docs). Use `async def` endpoints because LLM/vector-DB calls are I/O-bound — async lets one worker handle many concurrent waits.
- **Streaming:** Return tokens via Server-Sent Events (SSE) or `StreamingResponse` so users see output as it generates (better perceived latency).
- **Validation:** Pydantic models for request/response; enforce max prompt length, reject malformed input.
- **Resilience:** Timeouts, retries with exponential backoff (`tenacity`), circuit breakers, and a fallback model/response.
- **Rate limiting & auth:** API keys / OAuth2, per-user quotas, token budgets.
- **Observability:** Log prompt, model, token counts, latency, cost per request; trace with LangSmith / OpenTelemetry.

Q2. Sync vs async in Python — why does it matter for GenAI?

LLM calls spend most time waiting on the network. With sync code, a worker blocks the whole time. `asyncio` lets a single process fire many concurrent requests and process whichever returns first. Use `asyncio.gather` to fan out (e.g., query 5 chunks/agents in parallel). CPU-bound work (embedding math, tokenization at scale) belongs in a process pool, not the event loop.

Q3. What is the GIL and when does it bite you?

The Global Interpreter Lock allows only one thread to execute Python bytecode at a time. It hurts CPU-bound multithreading. Workarounds: `multiprocessing`, or offload to native libs (NumPy, PyTorch) that release the GIL. For our I/O-bound LLM services the GIL is rarely the bottleneck — async is the right tool.

Q4. Common Python questions to be ready for

- **Decorators** — wrap functions (e.g., a `@retry` or `@timed` decorator; used heavily for caching/auth).
- **Generators / `yield`** — lazy evaluation, streaming tokens, processing large files without loading into memory.
- **Context managers (`with`)** — resource cleanup (DB connections, file handles).
- **@dataclass vs Pydantic BaseModel** — dataclass = lightweight internal structs; Pydantic = validation + serialization at API boundaries.
- **List/dict comprehensions, `*args/**kwargs`, mutable default argument gotcha** (`def f(x=[])` bug).
- **Typing** — `typing / mypy` for maintainable code; important for team codebases.

Q5. How do you handle a slow / large data processing job in Python?

Batch it, stream it (generators), parallelize with `multiprocessing` or `concurrent.futures`, use vectorized NumPy/pandas ops, and cache repeated work. For embeddings: batch documents (e.g., 100 at a time) to the embedding API to amortize overhead and respect rate limits.

PART 2 — LLM Fundamentals

Q6. How does a transformer / LLM actually work (at a high level)?

Text is **tokenized** into subword units, each mapped to an **embedding** vector plus positional info. Stacked **self-attention** layers let every token weigh the relevance of every other token (Q/K/V — query, key, value matrices), producing context-aware representations. The model is **autoregressive**: it predicts the next token's probability distribution, samples one, appends it, and repeats. "Attention Is All You Need" (2017) introduced the architecture; decoder-only models (GPT family) dominate generative use.

Q7. Key generation/decoding parameters

- **Temperature** — randomness. 0 = deterministic/factual; higher = creative. Use low temp for RAG/extraction, higher for brainstorming.
- **Top-p (nucleus sampling)** — sample from the smallest set of tokens whose cumulative probability $\geq p$.
- **Top-k** — sample from k most likely tokens.
- **Max tokens** — output length cap (and cost cap).
- **Frequency / presence penalty** — reduce repetition.
- **Stop sequences** — halt generation at a marker.

Q8. What is the context window and why does it matter?

The max number of tokens (input + output) the model can attend to at once. It bounds how much context/RAG content/history you can feed. Exceeding it causes truncation or errors. Implications: chunking strategy, conversation-history summarization, and cost (you pay per token). Bigger windows aren't free — cost, latency, and "lost in the middle" attention degradation all grow.

Q9. Fine-tuning vs RAG vs prompt engineering — when to use which?

Approach	Use when	Cost/effort
Prompt engineering	Behavior achievable with instructions + few-shot examples	Lowest
RAG	Need fresh/proprietary/factual knowledge; reduce hallucination; citations	Medium
Fine-tuning	Need a specific style/format/behavior, latency/cost reduction via smaller model, or domain jargon	High
Rule of thumb: RAG for knowledge, fine-tuning for behavior . Start with prompting → RAG → fine-tune only if needed. They combine (fine-tune a model that also does RAG).		

Q10. What is PEFT / LoRA?

Parameter-Efficient Fine-Tuning. Instead of updating all billions of weights, **LoRA** freezes the base model and trains small low-rank adapter matrices injected into attention layers — far less compute/memory, and adapters are swappable. **QLoRA** adds 4-bit quantization of the base model so you can fine-tune on a single GPU. This is the practical way most teams fine-tune today.

Q11. What causes hallucinations and how do you reduce them?

Causes: model predicts plausible-sounding tokens without grounding; missing/outdated knowledge; ambiguous prompts. Mitigations: **RAG grounding with citations**, instruct "answer only from context, else say 'I don't know'", lower temperature, structured output constraints, self-verification/critique steps, and **evaluation** (faithfulness metrics). You reduce, never fully eliminate.

Q12. Quantization — what and why?

Reducing weight precision (FP16 → INT8/INT4) to shrink memory and speed inference, with small accuracy loss. Enables running larger models on smaller hardware. Related: distillation (train a small model to mimic a big one), pruning.

PART 3 — RAG (Retrieval-Augmented Generation) — HIGH PRIORITY

Q13. Walk me through a RAG pipeline end to end.

Ingestion (offline): 1. **Load** documents (PDF, HTML, DB, etc.). 2. **Chunk** them (split into passages). 3. **Embed** each chunk into a vector. 4. **Index / store** vectors + metadata in a vector DB.

Query (online): 5. **Embed the user query** with the same embedding model. 6. **Retrieve** top-k most similar chunks (vector similarity, often + metadata filters). 7. (Optional) **Re-rank** retrieved chunks with a cross-encoder. 8. **Augment** the prompt: stuff retrieved context + question into a template. 9. **Generate** the answer with the LLM, ideally with **citations**.

Q14. Chunking strategies — how do you choose chunk size?

- **Fixed-size** with overlap (e.g., 500-1000 tokens, 10-20% overlap) — simple baseline; overlap preserves context across boundaries.
- **Recursive character splitting** — split on paragraph → sentence → word boundaries to keep semantic units intact (`LangChain RecursiveCharacterTextSplitter`).
- **Semantic chunking** — split where embedding similarity drops (topic shifts).
- **Document-structure-aware** — by headings/sections for Markdown/HTML. Trade-off: **too small** = fragmented context, retrieval misses meaning; **too large** = noise, wasted tokens, diluted embeddings. Tune empirically against an eval set. Store rich **metadata** (source, page, section, date) for filtering and citations.

Q15. What are embeddings and how does similarity search work?

An embedding maps text to a dense vector where semantic similarity $\approx$ geometric closeness. Search computes distance between the query vector and stored vectors — usually **cosine similarity** (angle, magnitude-invariant — the standard for text), sometimes dot product or Euclidean. Nearest neighbors = most relevant chunks. Models: OpenAI `text-embedding-3-*`, `sentence-transformers` (all-MiniLM, BGE, E5), Cohere embed. **Always embed queries and documents with the same model.**

Q16. Exact vs approximate nearest neighbor (ANN) — HNSW?

Exact kNN is $O(N)$ — too slow at scale. **ANN** trades a little recall for huge speed. **HNSW (Hierarchical Navigable Small World)** builds a multi-layer graph you traverse greedily — the default in most vector DBs (fast, high recall, memory-heavy). Others: **IVF** (cluster into cells, search nearest cells), **PQ (Product Quantization)** to compress vectors. FAISS supports Flat (exact), IVF, HNSW, PQ combos.

Q17. How do you improve a RAG system that gives poor answers?

Diagnose **retrieval vs generation** first (log the retrieved chunks). - **Retrieval fixes:** better chunking, better embedding model, **hybrid search** (dense + BM25 keyword), metadata filtering, increase k, add a **re-ranker** (cross-encoder like Cohere Rerank / bge-reranker), **query rewriting/expansion**, **HyDE** (generate a hypothetical answer, embed that). - **Generation fixes:** better prompt ("use only context"), citation enforcement, larger/better model, context ordering (put key chunks first/last to fight "lost in the middle"). - **Systemic:** build an **eval set** and measure.

Q18. Dense vs sparse vs hybrid retrieval?

- **Sparse (BM25/keyword)** — great for exact terms, IDs, rare jargon; no semantic understanding.
- **Dense (embeddings)** — semantic/synonym matching; can miss exact keywords.
- **Hybrid** — run both, fuse scores (e.g., **Reciprocal Rank Fusion**). Best real-world recall. Weaviate, Pinecone, Qdrant, Elastic support hybrid natively.

Q19. What is re-ranking and why add it?

Retrieval (bi-encoder) is fast but coarse. A **cross-encoder re-ranker** takes each (query, chunk) pair together and scores true relevance — more accurate but slower. Pattern: retrieve top-50 cheaply → re-rank → keep top-5 for the prompt. Big quality boost for modest latency.

Q20. How do you evaluate a RAG system?

- **Retrieval metrics:** context precision, context recall, hit rate, MRR.
- **Generation metrics: faithfulness** (is the answer grounded in context — anti-hallucination), **answer relevancy**, correctness vs ground truth.
- **Tools: RAGAS**, TruLens, LangSmith/DeepEval; **LLM-as-a-judge** for scale.
- Build a **golden Q&A eval set**, run it in CI on every change, track regressions.

Q21. Advanced RAG patterns to name-drop

- **Parent-document retrieval** (embed small chunks, return larger parent for context).
- **Multi-query / RAG-Fusion** (generate query variants, merge results).
- **Self-RAG / CRAG** (model decides whether/what to retrieve, grades relevance).
- **GraphRAG** (knowledge-graph-augmented retrieval for multi-hop reasoning).
- **Contextual retrieval** (Anthropic — prepend chunk-level context before embedding).
- **Agentic RAG** (an agent iteratively retrieves, reasons, re-queries).

PART 4 — Vector Databases

Q22. Compare FAISS, Pinecone, and Weaviate.

Type	Library (Meta)	Managed cloud service	Open-source DB (self-host or cloud)
Persistence	You manage (in-memory/disk)	Fully managed, durable	Managed, durable
Scaling/ops	You build it	Zero-ops, auto-scale	You run it (or their cloud)
Metadata filter	Limited	Yes	Yes (rich, GraphQL)
Hybrid search	No (build yourself)	Yes	Yes (built-in)
Best for	Prototyping, full control, on-prem, cost	Production, no infra team, fast time-to-market	Open-source, hybrid search, built-in modules

Also mention **Qdrant**, **Milvus**, **Chroma (great for local dev)**, **pgvector (Postgres extension — reuse existing DB)**.

Q23. How do you choose a vector DB?

Consider: scale (millions vs billions of vectors), managed vs self-hosted (team's ops capacity), metadata filtering needs, hybrid search, latency/QPS SLAs, cost, cloud alignment (Azure AI Search if you're on Azure), and whether you already have Postgres (pgvector avoids a new system).

Q24. What metadata do you store and why?

Source URI, document ID, page/section, timestamp, author, access-control tags, chunk index. Enables: **citations**, **filtering** (only 2024 docs, only this tenant), **security** (row/tenant-level access), and **freshness** management.

Q25. How do you keep the index fresh / handle updates?

Incremental upserts keyed by document ID; delete-and-reindex on change; scheduled re-embedding jobs; track document versions; use TTL/timestamps to expire stale data. For large corpora, event-driven pipelines (a doc changes → message queue → re-embed → upsert).

PART 5 — Agentic Workflows (LangGraph, CrewAI, AutoGen) — HIGH PRIORITY

Q26. What is an "agent" in the LLM sense?

An LLM that can **reason, plan, and take actions via tools** in a loop, deciding the next step based on observations — rather than a single prompt→response. Core loop = **ReAct** (Reason + Act): Thought → Action (tool call) → Observation → repeat until done. Components: LLM (the reasoner), **tools** (functions/APIs it can call), **memory** (short + long term), and a **planner/controller**.

Q27. LangChain vs LangGraph — what's the difference?

- **LangChain** = building blocks + high-level chains (prompt templates, LLM wrappers, retrievers, output parsers, tool/agent abstractions). Great for linear/simple flows.
- **LangGraph** = low-level orchestration as a **stateful graph** (nodes = functions/agents, edges = transitions, plus a shared state object). Gives you **cycles, branching, conditional edges, human-in-the-loop, checkpointing/persistence, and controllability** — essential for reliable production agents where you need explicit control over flow, retries, and state. LangChain's older AgentExecutor is a black box; LangGraph makes the control flow explicit.

Q28. When would you use LangGraph over a simple chain?

When the workflow has **loops** (retry until valid), **branching** (route by intent), **multiple collaborating agents, human approval steps**, or needs **durable state/checkpointing** so a long-running workflow can pause/resume/recover. Example: an agent that retrieves → reasons → decides it needs more info → re-retrieves (a cycle a linear chain can't express).

Q29. Compare LangGraph, CrewAI, and AutoGen.

- **LangGraph** — graph-based, maximal **control and determinism**, explicit state; best for production workflows where reliability matters. Steeper learning curve.
- **CrewAI** — **role-based** multi-agent teams: you define agents with roles/goals/backstories, assign tasks, orchestrate sequential or hierarchical processes. High-level, intuitive, fast to prototype "a crew of collaborating specialists."
- **AutoGen** (Microsoft) — **conversational** multi-agent framework; agents talk to each other (and a user proxy) to solve tasks, strong for code-gen/execution and flexible conversation patterns. One-liner: LangGraph for control, CrewAI for role-based teams, AutoGen for conversational agent collaboration.

Q30. What are tools / function calling and how do agents use them?

Tools are functions with a **name, description, and typed schema** (JSON). The model, given the schema, decides when to call one and emits structured arguments; your code executes it and returns the result, which the model reads to continue. This is how agents search the web, query DBs, run code, hit APIs. Good tool **descriptions and schemas are critical** — the model routes based on them. (Anthropic/OpenAI both support native tool use; MCP standardizes tool/context exposure.)

Q31. How do you handle agent memory?

- **Short-term** — conversation history in the context window (trim/summarize when it overflows).
- **Long-term** — persist to a vector store (semantic recall of past interactions/facts) or a DB; retrieve relevant memories per turn.
- **Working/state memory** — LangGraph state object carries intermediate results across nodes.
- Summarize old turns to control cost/context.

Q32. What are the failure modes of agents and how do you control them?

- **Infinite loops / not terminating** → max-iteration caps, step budgets.
- **Hallucinated/invalid tool calls** → schema validation, retries, guardrails.
- **Cost/latency blowup** → limit tool calls, use cheaper models for sub-steps, cache.
- **Compounding errors** in multi-step chains → verification/critic steps, human-in-the-loop for high-stakes actions.
- **Unpredictability** → prefer explicit graphs (LangGraph) over fully autonomous agents; constrain the action space. "Give the agent as little autonomy as the task requires."

Q33. Multi-agent design — when and how?

Use multiple agents when tasks decompose into **specialized roles** (researcher, coder, reviewer) or need separation of concerns / parallelism. Patterns: **supervisor/orchestrator** routing to workers, **hierarchical** crews, **sequential pipeline**, **debate/critique**. Trade-off: more agents = more cost, latency, and coordination complexity — don't over-engineer; a single well-tooled agent often beats a fragile swarm.

PART 6 — NLP Fundamentals

Q34. Explain the core NLP tasks in the JD.

- **Summarization** — condense text. Extractive (pick key sentences) vs abstractive (generate new phrasing, what LLMs do). Watch context limits → **map-reduce** or **refine** summarization for long docs.
- **Q&A** — extractive (span from a passage) vs generative (RAG-style, synthesized with citations).
- **Chatbots** — multi-turn dialogue with memory, intent handling, and grounding.
- **Semantic search** — embedding-based retrieval by meaning, not keywords (the retrieval half of RAG).
- **Entity extraction (NER)** — pull structured entities (names, dates, orgs, amounts). Classic: spaCy/BERT token classification. Modern: LLM with a structured-output (JSON) schema.

Q35. How do you get structured output (e.g., JSON) reliably from an LLM?

- **Native structured output / function calling** with a JSON schema (most reliable).
- **Pydantic + instructor / LangChain output parsers** to validate and auto-retry on parse failure.
- **Constrained decoding / grammars** (e.g., Outlines) to force valid JSON.
- Low temperature, explicit schema in the prompt, few-shot examples. Always validate and handle failures gracefully.

Q36. Long-document summarization approaches?

- **Stuff** — everything in one prompt (only if it fits).
- **Map-reduce** — summarize chunks in parallel, then summarize the summaries.
- **Refine** — iteratively update a running summary chunk by chunk (better coherence, slower/sequential).

Q37. Classic NLP concepts you might be quizzed on

Tokenization (BPE/WordPiece/SentencePiece), stemming vs lemmatization, stop words, TF-IDF, BM25, word2vec/GloVe (static embeddings) vs contextual embeddings (BERT), encoder (BERT — understanding) vs decoder (GPT — generation) vs encoder-decoder (T5 — translation/seq2seq). BERT is bidirectional (masked LM); GPT is autoregressive.

Q38. How do you evaluate NLP outputs?

- Summarization/translation: ROUGE, BLEU, BERTScore (+ human eval).
 - Classification/NER: precision, recall, F1.
 - Generative/RAG: LLM-as-a-judge, faithfulness/relevancy (RAGAS), human review.
 - Semantic search: recall@k, MRR, nDCG. Metrics guide, but **human eval and task-specific golden sets** remain gold standard.
-

PART 7 — Azure AI Foundry / Google ADK & Cloud

Q39. What is Azure AI Foundry?

Microsoft's unified platform (evolution of Azure AI Studio) to **build, evaluate, deploy, and govern** GenAI apps. Provides: a **model catalog** (Azure OpenAI GPT models, plus open models — Llama, Mistral, etc.), **prompt flow** (visual + code orchestration/eval of LLM flows), **Azure AI Search** for RAG (managed vector + hybrid search), **evaluation & content safety**, **fine-tuning**, and enterprise **security/governance/private networking**. It's the "one place" to go from prototype to governed production on Azure.

Key Azure GenAI services to know: Azure OpenAI Service, Azure AI Search (vector/hybrid retrieval — the go-to RAG store on Azure), Azure AI Content Safety, Azure Machine Learning (MLOps), Azure Functions/Container Apps/AKS for hosting.

Q40. What is Google ADK (Agent Development Kit)?

Google's open-source **framework for building, evaluating, and deploying AI agents**, optimized for Gemini and the Google Cloud/**Vertex AI** ecosystem (but model-agnostic). Provides agent primitives, multi-agent orchestration, tools, and deployment paths (e.g., Vertex AI Agent Engine). Analogous role to LangGraph/CrewAI but Google-native. Related Google terms: **Vertex AI** (their ML/GenAI platform), **Gemini** models, **Vertex AI Search** for RAG.

Be honest about which one you've touched. If neither deeply: "I've worked primarily with [Azure OpenAI / OpenAI + LangChain]; Foundry/ADK are the platform layer that packages catalog + orchestration + eval + governance, and I'd ramp quickly since the concepts map directly to what I've built."

Q41. How do you deploy a GenAI app cloud-natively?

Containerize (Docker) → orchestrate (Kubernetes/AKS/GKE or serverless: Azure Container Apps, AWS Lambda/Fargate, Cloud Run). Externalize config/secrets (Key Vault / Secrets Manager). Autoscale on load. Put a gateway in front for auth, rate limiting, and routing. Use managed model endpoints (Azure OpenAI, Bedrock, Vertex) rather than self-hosting unless there's a reason. Add monitoring, logging, and cost dashboards.

PART 8 — MLOps / LLMOps, Versioning & CI/CD

Q42. What is LLMOps and how does it differ from MLOps?

MLOps = lifecycle mgmt for ML models (data → train → deploy → monitor). **LLMOps** adds LLM-specific concerns: **prompt management & versioning**, **RAG/retrieval eval**, **token cost & latency monitoring**, **hallucination/quality eval**, **guardrails & safety**, and often you're consuming an API rather than training. Non-determinism makes testing harder — you evaluate distributions/quality, not exact matches.

Q43. How do you version things in a GenAI system?

- **Models** — pin model + version (`gpt-4o-2024-xx`); track in a registry (MLflow, Azure ML registry).
- **Prompts** — treat as code: version in Git or a prompt registry, with eval scores per version.
- **Data/embeddings** — version the corpus and **re-embed when you change the embedding model** (embeddings from different models aren't comparable — a common gotcha).
- **Fine-tunes/adapters** — versioned artifacts with training data lineage.

Q44. What does CI/CD look like for LLM apps?

- **CI**: lint/type-check, unit tests for tools/parsers, and crucially an **evaluation gate** — run the golden eval set (RAGAS/LLM-judge) and fail the build if quality regresses beyond a threshold.
- **CD**: containerize, deploy via canary/blue-green, **A/B test** prompt/model changes, monitor, and roll back on regression.
- Prompt/model changes go through the same PR + eval pipeline as code.

Q45. How do you monitor a GenAI app in production?

- **Ops**: latency, throughput, error rates, token usage, **cost per request/user**.
- **Quality**: sampled LLM-as-a-judge scoring, user feedback (thumbs up/down), hallucination/faithfulness checks.
- **Drift**: changing input distributions, retrieval quality decay as corpus grows.
- **Safety**: content-safety filters, PII detection, jailbreak attempts.
- **Tracing**: end-to-end traces (LangSmith, Langfuse, OpenTelemetry) to debug multi-step chains/agents.

Q46. How do you control LLM cost and latency in production?

- **Model routing** — cheap/small model for easy tasks, big model only when needed.
- **Prompt caching** (Anthropic/OpenAI) and **semantic caching** of repeated queries.
- **Streaming** for perceived latency; **batching** embeddings.
- Trim context, limit max tokens, cap agent iterations.
- Set token **budgets/quotas** per user; monitor and alert on spend.

Q47. Security & safety concerns specific to LLM apps

- **Prompt injection** (esp. indirect, via retrieved/tool content) — treat model output as untrusted, sandbox tool execution, least-privilege tool permissions, validate before acting.
- **Data leakage / PII** — redaction, tenant isolation, don't send sensitive data to models that train on it.
- **Jailbreaks** — system-prompt hardening + input/output guardrails (Azure Content Safety, Llama Guard, NeMo Guardrails).
- **Access control** — filter retrieval by user permissions (don't retrieve docs a user can't see).
- **Hallucination in high-stakes** — human-in-the-loop, citations. OWASP maintains a **Top 10 for LLM Applications** — worth naming.

PART 9 — System Design Scenario (be ready to whiteboard)

Q48. "Design a chatbot over our company's internal documents."

Structure your answer: 1. **Requirements** — # docs, formats, update frequency, # users/QPS, latency SLA, accuracy needs, access control, on-prem vs cloud, budget. 2. **Ingestion pipeline** — connectors → parse → chunk (recursive, with overlap) → embed (batched) → upsert to vector DB with metadata + ACL tags. Event-driven for updates. 3. **Retrieval** — hybrid search (dense + BM25) → metadata/ACL filter → re-rank → top-k. 4. **Generation** — prompt template ("answer only from context, cite sources"), LLM via managed endpoint, streaming response, citations. 5. **Conversation** — memory with history summarization; multi-turn query rewriting (resolve "it/that"). 6. **Serving** — FastAPI async service, containerized, autoscaled, gateway (auth, rate limit). 7. **Eval & monitoring** — golden set + RAGAS in CI; log cost/latency/feedback; content safety. 8. **Guardrails** — injection defense, PII redaction, ACL-filtered retrieval. 9. **Trade-offs** — managed vs self-hosted, cost vs quality (model choice), latency vs re-ranking.

Talk requirements and trade-offs out loud — that's what "senior" is scored on.

Q49. "How would you take a research prototype (a notebook) to production?"

Refactor notebook → modular, tested Python package; externalize config/secrets; wrap in an API; add retries/timeouts/error handling; build eval set + CI gate; containerize + deploy with autoscaling; add monitoring/logging/cost tracking; add guardrails; document and set up rollback. Emphasize **reproducibility, evaluation, and observability** — the gap between "works in a demo" and "works for 10k users."

PART 9B — Python Coding Questions (live-coding / screen-share ready)

EPAM technical rounds usually include 2–4 hands-on coding problems. Expect a mix of (a) core Python/DSA, (b) "Pythonic" idioms, and (c) GenAI-flavored coding. Talk through your approach, complexity, and edge cases as you type.

PC1. Reverse words in a sentence (in place-ish)

```
def reverse_words(s: str) -> str:
    return " ".join(s.split()[::-1])
# "the quick brown fox" -> "fox brown quick the"
# .split() with no args collapses multiple spaces – mention that edge case.
```

PC2. Check if two strings are anagrams

```
from collections import Counter
def is_anagram(a: str, b: str) -> bool:
    return Counter(a) == Counter(b) # O(n); Counter handles frequency in one pass
```


PC3. Find the first non-repeating character

```
from collections import Counter
def first_unique(s: str) -> str | None:
    counts = Counter(s)
    for ch in s:
        # dict preserves insertion order (Py 3.7+)
        if counts[ch] == 1:
            return ch
    return None
```

PC4. Two Sum (classic — hash map, $O(n)$)

```
def two_sum(nums: list[int], target: int) -> list[int]:
    seen = {}
    # value -> index
    for i, n in enumerate(nums):
        if target - n in seen:
            return [seen[target - n], i]
        seen[n] = i
    return []
# Naive is  $O(n^2)$  nested loops; the hash map trades space for  $O(n)$  time. Say this out loud.
```

PC5. Count word frequency in a text (NLP-flavored)

```
from collections import Counter
import re
def word_freq(text: str, top_n: int = 5) -> list[tuple[str, int]]:
    words = re.findall(r"\b\w+\b", text.lower()) # tokenize + normalize case
    return Counter(words).most_common(top_n)
```

PC6. Remove duplicates preserving order

```
def dedupe(items: list) -> list:
    seen = set()
    return [x for x in items if not (x in seen or seen.add(x))]
# set() lookup is  $O(1)$ ; set(items) alone loses order — mention the trade-off.
```

PC7. Flatten a nested list (recursion)

```
def flatten(lst):
    for item in lst:
        if isinstance(item, list):
            yield from flatten(item) # generator + recursion
        else:
            yield item
# list(flatten([1, [2, [3, 4]], 5])) -> [1, 2, 3, 4, 5]
```

PC8. Fibonacci with memoization (shows caching knowledge)

```
from functools import lru_cache
@lru_cache(maxsize=None)
def fib(n: int) -> int:
    return n if n < 2 else fib(n - 1) + fib(n - 2)
# Without memoization it's  $O(2^n)$ ; lru_cache makes it  $O(n)$ . Relate to prompt/semantic caching.
```

PC9. Chunk a list into batches (directly relevant to embeddings/RAG)

```
def batch(items: list, size: int):
    for i in range(0, len(items), size):
        yield items[i:i + size]
# Used to batch documents to an embedding API to respect rate limits. Great to tie to your GenAI work.
```

PC10. Sliding window — longest substring without repeating chars

```
def longest_unique_substring(s: str) -> int:
    seen = {}
    start = best = 0
    for i, ch in enumerate(s):
        if ch in seen and seen[ch] >= start:
            start = seen[ch] + 1      # move window past the last occurrence
        seen[ch] = i
        best = max(best, i - start + 1)
    return best                      # O(n) sliding window
```

PC11. A retry decorator (production Python — they love this for API resilience)

```
import time
from functools import wraps

def retry(times: int = 3, delay: float = 1.0, backoff: float = 2.0):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            attempt, wait = 0, delay
            while True:
                try:
                    return fn(*args, **kwargs)
                except Exception:
                    attempt += 1
                    if attempt >= times:
                        raise
                    time.sleep(wait)
                    wait *= backoff      # exponential backoff — key for LLM/vector-DB calls
            return wrapper
        return decorator
```

PC12. Cosine similarity from scratch (VERY likely for a GenAI role)

```
import numpy as np

def cosine_similarity(a, b) -> float:
    a, b = np.array(a), np.array(b)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))
# This is the core of semantic search / vector retrieval. Be ready to explain WHY cosine
# (magnitude-invariant — compares direction/meaning, not length) vs Euclidean.
```

PC13. Top-k most similar vectors (mini vector-search)

```
import numpy as np

def top_k(query, corpus: list, k: int = 3) -> list[int]:
    sims = [cosine_similarity(query, v) for v in corpus]
    return list(np.argsort(sims)[::-1][:k])    # indices of k highest scores
# This is literally what FAISS/Pinecone do at scale (with ANN indexes like HNSW).
```

PC14. Common Python "gotcha" questions

```
# 1) Mutable default argument bug:
def bad(x, acc=[]):      # acc is created ONCE, shared across calls -> accumulates!
    acc.append(x); return acc
def good(x, acc=None):
    acc = acc if acc is not None else []    # correct fix
    acc.append(x); return acc

# 2) Shallow vs deep copy:
import copy
copy.copy(obj)          # shallow – nested objects shared
copy.deepcopy(obj)      # fully independent clone

# 3) is vs == : 'is' checks identity (same object); '==' checks value equality.

# 4) List comprehension vs generator: [x for x in xs] builds a list in memory;
#     (x for x in xs) is lazy – use generators for large/streaming data.
```

PART 9C — SQL Questions

Even for a GenAI role, EPAM screens SQL fundamentals (metadata stores, analytics, pgvector). Know joins, aggregation, GROUP BY/HAVING, window functions, and subqueries cold.

Assume these tables: employees(id, name, department_id, salary, manager_id, hire_date) departments(id, name) orders(id, customer_id, amount, order_date)

SQL1. 2nd highest salary (asked constantly)

```
-- Method A: window function (best, handles ties clearly)
SELECT DISTINCT salary
FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees) t
WHERE rnk = 2;

-- Method B: subquery
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Be ready to explain **RANK vs DENSE_RANK vs ROW_NUMBER**: RANK skips numbers after ties (1,2,2,4); DENSE_RANK doesn't (1,2,2,3); ROW_NUMBER is always unique (1,2,3,4).

SQL2. Highest-paid employee per department (window function)

```
SELECT name, department_id, salary
FROM (
    SELECT name, department_id, salary,
           RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rnk
    FROM employees
) t
WHERE rnk = 1;
```

PARTITION BY = restart the ranking within each group — the key window-function concept.

SQL3. Count employees per department, only departments with >5 (WHERE vs HAVING)

```
SELECT d.name, COUNT(e.id) AS headcount
FROM departments d
LEFT JOIN employees e ON e.department_id = d.id
GROUP BY d.name
HAVING COUNT(e.id) > 5;
```

WHERE filters rows before grouping; HAVING filters groups after aggregation. LEFT JOIN keeps departments with zero employees.

SQL4. Explain the JOIN types

- **INNER JOIN** — only matching rows in both tables.
- **LEFT (OUTER) JOIN** — all left rows + matches (NULLs where none).
- **RIGHT JOIN** — mirror of left.
- **FULL OUTER JOIN** — all rows from both, NULLs where no match.
- **CROSS JOIN** — Cartesian product.
- **SELF JOIN** — table joined to itself (e.g., employee → manager below).

SQL5. Self-join — list each employee with their manager's name

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id;
```

SQL6. Find duplicate records

```
SELECT email, COUNT(*)
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

SQL7. Employees earning more than their department average (correlated subquery)

```
SELECT name, salary, department_id
FROM employees e
WHERE salary > (
    SELECT AVG(salary) FROM employees
    WHERE department_id = e.department_id -- correlated: runs per outer row
);
```

SQL8. Monthly order totals (aggregation + date functions)

```
SELECT DATE_TRUNC('month', order_date) AS month,
       COUNT(*) AS num_orders,
       SUM(amount) AS revenue
FROM orders
GROUP BY 1
ORDER BY 1;
```

SQL9. Running total (window function)

```
SELECT order_date, amount,
       SUM(amount) OVER (ORDER BY order_date
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM orders;
```

SQL10. Customers who never placed an order (anti-join)

```
SELECT c.id, c.name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NULL;          -- or: WHERE NOT EXISTS (...)
```

SQL11. Delete duplicate rows keeping the earliest

```
DELETE FROM users a
USING users b
WHERE a.id > b.id AND a.email = b.email;  -- keeps lowest id per email
```

SQL12. pgvector — semantic search in SQL (ties SQL to your GenAI story!)

```
-- With the pgvector extension, RAG retrieval can live in Postgres:
SELECT id, content, embedding <=> '[0.1, 0.2, ...]' AS distance
FROM documents
ORDER BY embedding <=> '[0.1, 0.2, ...]'  -- <=> is cosine distance
LIMIT 5;
-- Mention: pgvector lets you avoid a separate vector DB when you already run Postgres,
-- supports HNSW/IVFFlat indexes, and combines vector search with normal SQL filters (hybrid).
```

SQL13. Query-optimization talking points (they may ask "why is this query slow?")

- **Indexes** on JOIN/WHERE/ORDER BY columns (B-tree default; GIN/HNSW for special types).
- **EXPLAIN ANALYZE** to read the query plan (seq scan vs index scan).
- Avoid `SELECT *`; select only needed columns.
- Filter early; avoid functions on indexed columns in WHERE (breaks index use).
- Beware N+1 queries from the app layer — batch/join instead.
- Partitioning and materialized views for large analytics tables.

SQL14. ACID (concept check)

Atomicity (all-or-nothing txn), **C**onsistency (valid state → valid state), **I**solation (concurrent txns don't corrupt each other), **D**urability (committed data survives crashes). Contrast with NoSQL/BASE (eventual consistency) if asked SQL vs NoSQL.

PART 10 — Behavioral / Leadership (senior role → they WILL ask)

Use **STAR** (Situation, Task, Action, Result). Prepare 3–4 stories with metrics.

Q50. Likely behavioral questions

- **"Tell me about a GenAI project you led end to end."** → problem, your architecture decisions, trade-offs, outcome/metrics, what you'd do differently.
- **"A time you mentored engineers / led technically."** → the JD says mentoring teams and leading technical strategy. Have a concrete example (code reviews, design docs, onboarding, setting standards).
- **"Disagreement with a stakeholder / teammate."** → how you used data/prototypes to resolve it.
- **"A project that failed or a hard bug."** → ownership + learning.
- **"How do you keep up with the fast-moving GenAI field?"** → papers (arXiv), newsletters, building side projects, following Anthropic/OpenAI/Google releases.
- **"How do you translate a fuzzy research idea into a deployable product?"** → the JD explicitly lists this: start with a scoped POC + eval metric, de-risk the hard part first, then harden for scale/cost/reliability.
- **"Why EPAM / why this role?"** → global-scale engineering consultancy, diverse client problems, strong engineering culture, growth (certifications, tech talks).

PART 11 — Rapid-fire cheat sheet (know the one-liner)

- **Temperature 0** → deterministic/factual.
- **Cosine similarity** → standard text similarity metric.
- **HNSW** → default ANN graph index.
- **Chunk overlap** → preserves context across boundaries.
- **RAG** → knowledge; **fine-tuning** → behavior.
- **LoRA/QLoRA** → cheap fine-tuning via adapters + quantization.
- **ReAct** → reason + act tool loop.
- **LangGraph** → stateful graph, cycles, control; **CrewAI** → role-based teams; **AutoGen** → conversational agents.
- **Hybrid search** → dense + BM25 + RRF fusion.
- **Re-ranker** → cross-encoder, retrieve-many-then-rerank.
- **RAGAS** → faithfulness/relevancy/context metrics.
- **Faithfulness** → answer grounded in retrieved context (anti-hallucination).
- **Prompt injection** → #1 LLM security risk; treat retrieved/tool content as untrusted.
- **Embed queries & docs with the SAME model.** Re-embed everything if you change models.
- **Azure AI Search** → managed RAG store on Azure; **Vertex AI Search** → on Google.
- **Lost in the middle** → LLMs attend better to start/end of context.

PART 12 — Smart questions to ask THEM (end of interview)

- What does the GenAI team's current stack and roadmap look like?
- Are these client-facing solutions or internal platforms? What scale?
- How mature is the LLMops/eval tooling today, and where do you want it to go?
- What does success in the first 90 days look like for this role?
- How much of the role is hands-on building vs. technical leadership/mentoring?

Final tips

1. **Lead with your real projects** — anchor abstract answers in something you built.
2. **Think out loud** in design questions; state trade-offs — seniority is judged on judgment, not just facts.
3. **It's fine to say "I don't know"** + how you'd find out. Bluffing gets caught.
4. **Have numbers ready** — latency, cost, accuracy, corpus size, user counts from past work.
5. **Brush up on your weakest JD item tonight** (likely Azure Foundry OR Google ADK — pick the one closer to your experience and go one level deeper).

Good luck, Joemon. ☐